1. Schema Setup

```
CREATE TABLE Product (
	productID SERIAL PRIMARY KEY,
	productName VARCHAR(50) NOT NULL
);

CREATE TABLE Dealer (
	dealerID SERIAL PRIMARY KEY,
	dealerName VARCHAR(50) NOT NULL,
	region VARCHAR(50)
);

CREATE TABLE Sales (
	saleID SERIAL PRIMARY KEY,
	productID INT REFERENCES Product(productID),
	dealerID INT REFERENCES Dealer(dealerID),
	saleDate DATE NOT NULL,
	price NUMERIC(10,2) NOT NULL
);

INSERT INTO Product (productName) VALUES
('Laptop'),
('Tablet'),
('Phone');

INSERT INTO Dealer (dealerName, region) VALUES
('TechWorld', 'Midwest'),
('GigaStore', 'Southeast'),
('ElectroHub', 'Northeast');
```

```
INSERT INTO Sales (productId, dealerId, saleDate, price) VALUES
(1, 1, '2024-01-10', 1200.00),
(1, 1, '2024-01-15', 1300.00),
(1, 2, '2024-02-01', 1250.00),
(2, 1, '2024-01-12', 500.00),
(2, 3, '2024-03-05', 550.00),
(2, 3, '2024-03-07', 530.00),
(3, 2, '2024-02-10', 900.00),
(3, 2, '2024-02-11', 920.00),
(3, 3, '2024-03-20', 890.00),
(3, 1, '2024-01-22', 880.00),
(1, 3, '2024-03-18', 1280.00),
(2, 2, '2024-02-19', 510.00);
```

Problem 1 — Constructing the Fact Table

- SELECT
  - s.saleId,
  - p.productName,
  - d.dealerName,
  - d.region,
  - s.saleDate,
  - s.price
- FROM Sales s
- JOIN Product p ON s.productId = p.productId
- JOIN Dealer d ON s.dealerId = d.dealerId
- ORDER BY s.saleDate;

- This query joins the Sales fact table with the Product and Dealer dimension tables to produce a combined dataset. It shows each sale along with its product name, dealer name, region, date and price and returns the fully detailed transation-level rows.

```
-- Problem 1 Creating the fact table
SELECT
    s.saleId,
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1

	saleid integer	productname character varying (50)	dealername character varying (50)	region character varying (50)	saledate date	price numeric (10,2)
1	1	Laptop	TechWorld	Midwest	2022-01-...	1200.00
2	4	Tablet	TechWorld	Midwest	2022-01-...	500.00
3	2	Laptop	TechWorld	Midwest	2023-01-...	1300.00
4	10	Phone	TechWorld	Midwest	2023-01-...	880.00
5	7	Phone	GigaStore	Southeast	2023-02-...	900.00
6	5	Tablet	ElectroHub	Northeast	2023-03-...	550.00
7	3	Laptop	GigaStore	Southeast	2024-02-...	1250.00
8	12	Tablet	GigaStore	Southeast	2024-02-...	510.00
9	6	Tablet	ElectroHub	Northeast	2024-03-...	530.00
10	11	Laptop	ElectroHub	Northeast	2024-03-...	1280.00
11	8	Phone	GigaStore	Southeast	2025-02-...	920.00
12	9	Phone	ElectroHub	Northeast	2025-03-...	890.00

- These joins creates a fact table used for OLAP analysis with all dimensions (product, dealer, and date) and the measure of them (price). Every cube operation after builds aggregations from this base dataset.

Problem 2 — Applying the GROUP BY CUBE Operator

- SELECT
  p.productName,
  d.dealerName,
  s.saleDate,
  SUM(s.price) AS totalSales
  FROM Sales s
  JOIN Product p ON s.productId = p.productId
  JOIN Dealer d ON s.dealerId = d.dealerId
  GROUP BY CUBE (p.productName, d.dealerName, s.saleDate)
  ORDER BY p.productName, d.dealerName, s.saleDate;
- This query computes all possible aggregations of sales totals across the product, dealer and date dimensions. It returns detailed rows, subtotal rows, and rolled-up totals including product totals, dealer totals, date totals and the grand total.

```
63  -- Problem 2 Group by CUBE
```

Data Output Messages Notifications

Showing rows: 1 to 64 Page No:

	productname character varying (50)	dealername character varying (50)	saledate date	totalsales numeric
1	Laptop	ElectroHub	2024-03-...	1280.00
2	Laptop	ElectroHub	[null]	1280.00
3	Laptop	GigaStore	2024-02-...	1250.00
4	Laptop	GigaStore	[null]	1250.00
5	Laptop	TechWorld	2022-01-...	1200.00
6	Laptop	TechWorld	2023-01-...	1300.00
7	Laptop	TechWorld	[null]	2500.00
8	Laptop	[null]	2022-01-...	1200.00
9	Laptop	[null]	2023-01-...	1300.00
10	Laptop	[null]	2024-02-...	1250.00
11	Laptop	[null]	2024-03-...	1280.00
12	Laptop	[null]	[null]	5030.00
13	Phone	ElectroHub	2025-03-...	890.00
14	Phone	ElectroHub	[null]	890.00

Total rows: 64 Query complete 00:00:00.257 Rows selected: 64

- The GROUP BY CUBE operator automatically generates every combination of grouped and rolled up dimensions, producing a full multi-dimensional cube. This extends what normal grouping would do by creating all subtotal and total levels without writing multiple separate queries.

Problem 3 — Slice and Dice Operations

- -- Slice (only Laptop sales)
  SELECT
  d.dealerName,
  s.saleDate,
  SUM(s.price) AS totalSales
  FROM Sales s
  JOIN Product p ON s.productId = p.productId
  JOIN Dealer d ON s.dealerId = d.dealerId
  WHERE p.productName = 'Laptop'
  GROUP BY d.dealerName, s.saleDate
  ORDER BY s.saleDate;

- This query filters the cube to only include rows for a single product (Laptop). It then aggregates the remaining dimensions to show how that product performs across dealers and dates.

```sql
-- Problem 3 Slice and Dice
-- Slice (only Laptop sales)
SELECT
    d.dealerName,
    s.saleDate,
    SUM(s.price) AS totalSales
FROM Sales s
JOIN Product p ON s.productId = p.productId
JOIN Dealer d ON s.dealerId = d.dealerId
WHERE p.productName = 'Laptop'
GROUP BY d.dealerName, s.saleDate
ORDER BY s.saleDate;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page N

	dealername character varying (50)	saledate date	totalsales numeric
1	TechWorld	2022-01-...	1200.00
2	TechWorld	2023-01-...	1300.00
3	GigaStore	2024-02-...	1250.00
4	ElectroHub	2024-03-...	1280.00

Total rows: 4 Query complete 00:00:00.192

- A slice fixes one dimension to a single value, creating a smaller 2D sub-cube. This allows focused analysis on one attribute while still viewing aggregated patterns in the remaining dimensions.

- -- Dice (Dealers in Southeast or Northeast AND date range February-March)
```
SELECT
        p.productName,
        d.dealerName,
        d.region,
        s.saleDate,
        SUM(s.price) AS totalSales
FROM Sales s
JOIN Product p ON s.productId = p.productId
JOIN Dealer d ON s.dealerId = d.dealerId
WHERE d.region IN ('Southeast', 'Northeast')
 AND s.saleDate BETWEEN '2024-02-01' AND '2024-03-31'
GROUP BY p.productName, d.dealerName, d.region, s.saleDate
ORDER BY s.saleDate;
```

- This query restricts the dataset to specific regions (Southeast and Northeast) and a defined date range (February to March), returning sales totals only for the selected products, dealers, and dates. It shows a filtered multi-dimensional view of sales activity.

```sql
-- Dice (Dealers in Southeast or Northeast AND
-- date range February-March)
SELECT
    p.productName,
    d.dealerName,
    d.region,
    s.saleDate,
    SUM(s.price) AS totalSales
```

Data Output Messages Notifications

Showing rows: 1 to 8 Page N

	productname character varying (50)	dealername character varying (50)	region character varying (50)	saledate date	totalsales numeric
1	Laptop	GigaStore	Southeast	2024-02-...	1250.00
2	Phone	GigaStore	Southeast	2024-02-...	900.00
3	Phone	GigaStore	Southeast	2024-02-...	920.00
4	Tablet	GigaStore	Southeast	2024-02-...	510.00
5	Tablet	ElectroHub	Northeast	2024-03-...	550.00
6	Tablet	ElectroHub	Northeast	2024-03-...	530.00
7	Laptop	ElectroHub	Northeast	2024-03-...	1280.00
8	Phone	ElectroHub	Northeast	2024-03-...	890.00

Total rows: 8 Query complete 00:00:00.156

- A dice operation applied multiple dimension filters simultaneously, forming a smaller, more targeted sub-cube. This allows you to analyze a constrained portion of the cube across several dimensions at once.

Problem 4 — Persisting Aggregated Results

- CREATE MATERIALIZED VIEW sales_cube_mv AS
  SELECT
  p.productName,
  d.dealerName,
  s.saleDate,
  SUM(s.price) AS totalSales
  FROM Sales s
  JOIN Product p ON s.productId = p.productId
  JOIN Dealer d ON s.dealerId = d.dealerId
  GROUP BY CUBE (p.productName, d.dealerName, s.saleDate);

- This query stores the cube's aggregated results in a materialized view, capturing all products, dealers, regions, and dates at the time it is created. The stored results can be viewed without re-running the full cube computation

- -- Before Refresh
  SELECT * FROM sales_cube_mv
  ORDER BY productName, dealerName, saleDate;

```
117 -- Before Refresh
```

Data Output Messages Notifications

Showing rows: 1 to 64 Page No:

	productname character varying (50)	dealername character varying (50)	saledate date	totalsales numeric
1	Laptop	ElectroHub	2024-03-...	1280.00
2	Laptop	ElectroHub	[null]	1280.00
3	Laptop	GigaStore	2024-02-...	1250.00
4	Laptop	GigaStore	[null]	1250.00
5	Laptop	TechWorld	2024-01-...	1200.00
6	Laptop	TechWorld	2024-01-...	1300.00
7	Laptop	TechWorld	[null]	2500.00
8	Laptop	[null]	2024-01-...	1200.00
9	Laptop	[null]	2024-01-...	1300.00
10	Laptop	[null]	2024-02-...	1250.00
11	Laptop	[null]	2024-03-...	1280.00
12	Laptop	[null]	[null]	5030.00
13	Phone	ElectroHub	2024-03-...	890.00

Total rows: 64 Query complete 00:00:00.609

- Shows the tables before being inserted into and refreshed to show new values.

INSERT INTO Sales (productId, dealerId, saleDate, price) VALUES
(1, 3, '2025-04-02', 1400.00),
(2, 2, '2025-04-05', 540.00),
(3, 1, '2025-04-07', 930.00);

REFRESH MATERIALIZED VIEW sales_cube_mv;
SELECT * FROM sales_cube_mv
ORDER BY productName, dealerName, saleDate;

```
126 REFRESH MATERIALIZED VIEW sales_cube_mv;
127 SELECT * FROM sales_cube_mv
128 ORDER BY productName, dealerName, saleDate;
```

Data Output Messages Notifications

Showing rows: 1 to 76 Page No

	productname character varying (50)	dealername character varying (50)	saledate date	totalsales numeric
1	Laptop	ElectroHub	2024-03-...	1280.00
2	Laptop	ElectroHub	2025-04-...	1400.00
3	Laptop	ElectroHub	[null]	2680.00
4	Laptop	GigaStore	2024-02-...	1250.00
5	Laptop	GigaStore	[null]	1250.00
6	Laptop	TechWorld	2024-01-...	1200.00
7	Laptop	TechWorld	2024-01-...	1300.00
8	Laptop	TechWorld	[null]	2500.00
9	Laptop	[null]	2024-01-...	1200.00
10	Laptop	[null]	2024-01-...	1300.00
11	Laptop	[null]	2024-02-...	1250.00
12	Laptop	[null]	2024-03-...	1280.00

Total rows: 76 Query complete 00:00:00.222

- The materialized view after insert and refresh showing the new updated view.
- After inserting new rows into the Sales table, the REFRESH MATERIALIZED VIEW statement recomputes the cube using the updated data. This demonstrates how OLAP systems can materialize expensive summaries and then updates them as new facts arrive.